

ThoughtLens — Full Product Design Document

A Platform-Agnostic LLM Interpretability Toolkit Based on Anthropic's Attribution Graph Research

Version: 1.0 · **Status:** Pre-build specification · **Author:** Chandu Ch

Executive Summary

ThoughtLens is an open-source Python package and interactive web interface that exposes the internal computational circuits of any large language model — closed-source API models (GPT-4o, Claude) or locally-hosted weights (Llama, Mistral, Phi). It directly operationalises Anthropic's **Circuit Tracing** and **On the Biology of a Large Language Model** research papers (March 2025)[cite:21] by packaging their core methods into a tool any ML researcher or engineer can `pip install` and run within minutes.

The name **ThoughtLens** is available on PyPI[cite:17], is memorable, accurately describes the product (a lens into a model's thoughts), and carries none of the generic "LLM tool" noise of names like `llm-circuit` or `llm-tools`. It is the name this document will use throughout.

1. Name Candidates (All PyPI-Available)

The following names were verified against the PyPI package registry as of May 2026. All return HTTP 404 (unregistered):

Name	Rationale	Verdict
thoughtlens	"A lens into LLM thoughts" — metaphorically precise, memorable, brandable	✔ Recommended
synaptrace	Neural-scientific: "trace the synapses" — strong for the research audience	✔ Strong alternative
axonscope	"Scope" family, axon = neural pathway; good SEO within the interpretability space	✔ Good
cortexlens	Evokes "looking into the cortex" — slightly more clinical	✔ Good
cogniscope	"Cognition + scope" — broad but memorable	✔ Acceptable
thoughtscope	Direct cousin of ThoughtLens	✔ Fallback
glassbox-llm	Directly contrasts the "black-box" framing from the Anthropic announcement	✔ Good for marketing
neuronmap	Descriptive but slightly overlaps with existing neuroscience tools	✔ Acceptable

Final recommendation: `thoughtlens` — unique, instantly communicates the purpose, and works as both a package name and a product brand.

2. Research Foundation — What the Papers Actually Show

Before building anything, the implementation must be grounded exactly in what Anthropic demonstrated — not a reimplementaion of generic attention visualization or SHAP scores.

2.1 Paper 1: Circuit Tracing (Methods Paper)[cite:21]

The core innovation is the **Cross-Layer Transcoder (CLT)**:

- Standard MLP neurons are **polysemantic** — one neuron fires for dozens of unrelated concepts (the "superposition hypothesis"). This makes them uninterpretable.
- The CLT is a separate neural network trained to replace each MLP layer's output with a **sparse, monosemantic reconstruction**. It learns a dictionary of ~16,000–65,000 features per layer, where each feature fires strongly for a single interpretable concept.
- After training, every MLP output on every token can be expressed as a **sparse sum of features**, each with a human-interpretable label (e.g., "words following a capital city", "conjugations of 'to be'", "entities that are musicians").
- These features can be connected across layers into an **attribution graph** — a directed computational graph showing which earlier features caused which later features, ultimately tracing the full path from input token to output logit.

What makes this different from prior interpretability work:

- Prior work (TransformerLens, attention visualization) describes *what* information exists inside a model at a snapshot. Attribution graphs show *why* a specific token was predicted — a full causal story.
- The approach handles **cross-layer influence**, not just within-layer activations.
- Features are **interventionable**: you can clamp, amplify, or suppress a specific feature during inference and observe the downstream effect on the output. This is how Anthropic confirmed their interpretations weren't post-hoc rationalisations.

2.2 Paper 2: Biology of a Large Language Model (Case Studies)[cite:21]

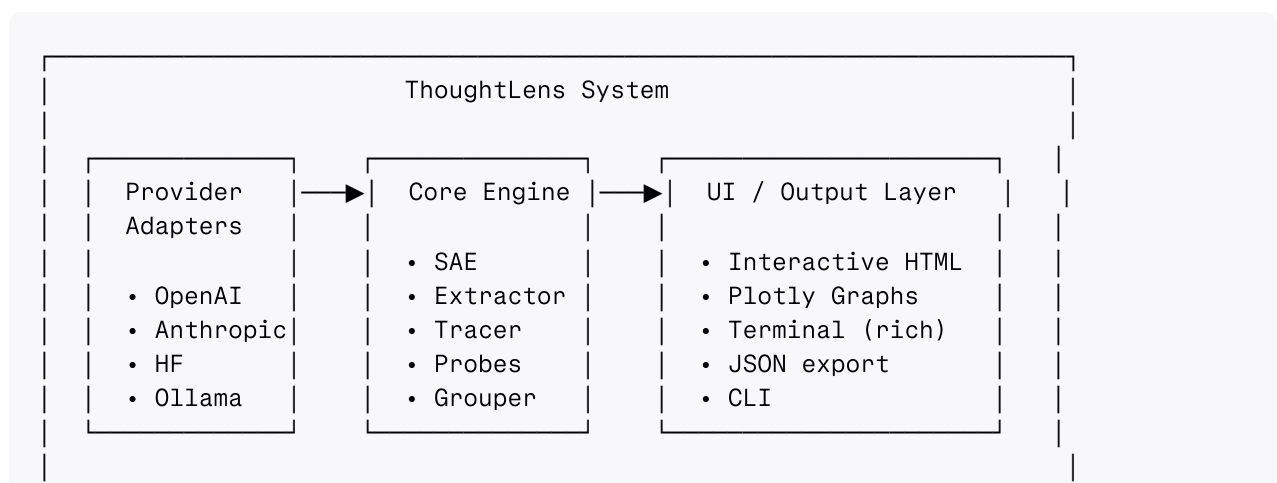
Applied to Claude 3.5 Haiku. The 10 case studies that ThoughtLens must implement as first-class probes:

#	Case Study	Key Finding	Probe Implementation
1	Multi-hop geography (Dallas→Texas→Austin)	Model maintains an intermediate "state" concept across two reasoning steps, each handled by distinct feature clusters	MultiHopProbe
2	Capitals of countries	Distinct circuits activate for major capitals vs. obscure capitals — the latter show higher uncertainty features	CapitalsProbe
3	Rhyme planning in poetry	Before writing line 2, rhyme candidates for the line-end activate in the residual stream. Model writes <i>backward</i> to reach a pre-planned endpoint	RhymePlanningProbe

#	Case Study	Key Finding	Probe Implementation
4	Emotional states / personas	Model maintains a distinct "self-concept" node that modulates tone and content, separate from factual retrieval	PersonaConsistencyProbe
5	Multilingual abstraction	Semantically identical concepts across languages share a language-independent abstract feature . The model translates <i>to</i> the output language only at the final step	MultilingualAbstractionProbe
6	Hallucination mechanism	A default "unknown entity" suppression feature fires when the model has low confidence. When it fails to fire on a partially-known name, the model confabulates	HallucinationProbe
7	Chain-of-thought faithfulness	When given a wrong hint, CoT scratchpad tokens sometimes encode the wrong answer first, then generate backwards-rationalised steps to justify it	CoTFaithfulnessProbe
8	Harmful content refusal	A "grammar continuation" feature can override a "safety refusal" feature when mid-sentence. The jailbreak anatomy: grammatical obligation > safety suppression	RefusalProbe
9	Suppression / self-censorship	A distinct "I should not say this" feature cluster inhibits output tokens even when the knowledge features are fully activated	SuppressorProbe
10	Motivated reasoning	When an incorrect answer is pre-supplied, the model activates the target answer's feature first, then constructs backwards reasoning in the CoT	MotivatedReasoningProbe

3. Product Architecture

3.1 High-Level System Design



```

ProviderOutput (normalised interface across all platforms)
{ tokens, activations[n_layers, n_tok, d], attentions, logits }

```

3.2 Module Map (Full)

```

thoughtlens/
├── __init__.py          ← Public API: Scope, Feature, AttributionGraph, Probe
├── scope.py             ← Main entry-point class
├── cli.py               ← `thoughtlens trace/probe/report` CLI
├──
├── providers/
│   ├── base.py         ← BaseProvider ABC + ProviderOutput dataclass
│   ├── openai.py       ← Black-box via logprobs API
│   ├── anthropic.py    ← Black-box via Messages API
│   ├── huggingface.py  ← White-box: real activations + attentions via hooks
│   ├── ollama.py       ← Black-box via Ollama REST API
│   └── registry.py     ← Provider factory
├──
├── features/
│   ├── feature.py      ← Feature, FeatureSet dataclasses
│   ├── sae.py          ← TopK Sparse Autoencoder (train / encode / decode)
│   ├── extractor.py    ← FeatureExtractor: white-box (SAE) + black-box (par
│   ├── intervention.py ← FeatureIntervention: amplify / inhibit / clamp
│   ├── labeler.py      ← Auto-labeling features via LLM explanation
│   └── cache.py        ← Activation caching for SAE training
├──
├── circuits/
│   ├── graph.py        ← AttributionGraph, CircuitNode, CircuitEdge
│   ├── tracer.py       ← CircuitTracer: builds attribution graph
│   ├── supernodes.py   ← SupernodeGrouper: clusters features
│   ├── paths.py        ← TopPathFinder: highest-weight causal paths
│   └── diff.py         ← GraphDiff: compare two attribution graphs
├──
├── probes/
│   ├── base.py         ← BaseProbe ABC
│   ├── runner.py       ← ProbeRunner + ProbeResult
│   ├── builtin/
│   │   ├── multi_hop.py ← Case study 1 + 2
│   │   ├── planning.py  ← Case study 3 (rhyme pre-planning)
│   │   ├── persona.py   ← Case study 4 (emotional consistency)
│   │   ├── multilingual.py ← Case study 5
│   │   ├── hallucination.py ← Case study 6
│   │   ├── cot_faithfulness.py ← Case study 7
│   │   ├── refusal.py   ← Case study 8 (jailbreak anatomy)
│   │   ├── suppressor.py ← Case study 9
│   │   └── motivated_reasoning.py ← Case study 10
│   └── custom.py       ← User-defined probe base class
├──
├── visualization/
│   ├── graph_viz.py    ← Plotly interactive attribution graph
│   ├── token_heatmap.py ← Token-level feature activation heatmap
│   ├── feature_browser.py ← Browse and filter features by layer/activation
│   ├── layer_stream.py ← Residual stream trajectory visualization
│   └── attention_viz.py ← Attention pattern overlay

```

```

├── report.py                ← Self-contained HTML report generator
├── utils/
│   ├── tokenizer_utils.py
│   ├── math_utils.py       ← Cosine sim, topk, sparse ops
│   └── colors.py           ← Consistent color scheme for all visualizations

```

4. Deep UI Layer Design — "What Is This LLM Really Thinking?"

This is the most important section. The UI must make the paper's abstract concepts **viscerally legible** to a working engineer — not just pretty graphs, but a navigation system for the model's mind.

4.1 The Five Core Views

Every UI surface — whether HTML report, notebook widget, or future web app — must support these five views, each answering a distinct interpretability question.

View 1: Token Feature Heatmap

Question: "Which tokens are activating what?"

A colour-coded token strip at the top of every trace, where each token cell is shaded by the count + magnitude of features active on it. Hovering a token shows a pop-up listing its top 5 active features with activation strengths.

```

Input:  "The  capital  of  the  state  containing  Dallas  is"
        [teal] [red]  [teal] [teal] [teal] [teal] [red] [teal]
        0.1   0.9   0.1  0.1   0.2   0.3   0.8   0.1
                        ↑
                        [city_entity: 0.91]
                        [US_state_member: 0.72]
                        [geographic_named_entity: 0.61]

```

Colour encoding:

- Deep teal = high feature activation (the model is "paying attention" to this token)
- Light beige = low / background activation
- Red overlay = a safety/refusal feature is active on this token
- Gold overlay = an uncertainty/hallucination-risk feature is active

Implementation: `token_heatmap.py` — outputs a `Plotly go.Heatmap` + annotation overlays.

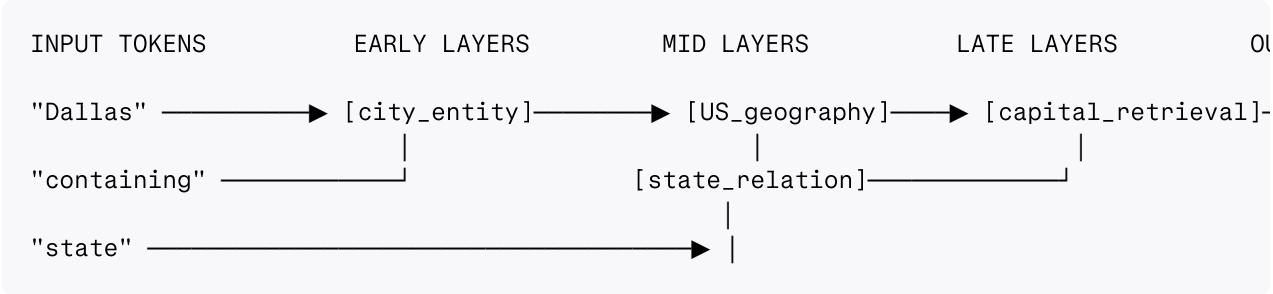
View 2: Attribution Graph (The Core View)

Question: "What caused the model to predict this token?"

A layered directed graph where:

- **Left column:** Input token nodes
- **Middle columns:** Feature supernodes, one column per transformer layer band (early / mid / late)
- **Right column:** Output token node

Edge width encodes attribution weight. Edge colour encodes polarity — teal for promoting edges, red for suppressing edges.



Node types with distinct visual treatment:

Node Type	Shape	Color	Meaning
Input token	Rounded rect	Warm beige	Raw input
Feature node	Circle	Teal gradient by activation	Interpretable concept
Supernode	Hexagon	Dark teal	Cluster of related features
Output token	Star	Gold	Predicted next token
Error term	Diamond	Muted red	Unexplained residual
Safety node	Shield	Orange	Harm-related feature

Interactions (web UI phase):

- Click a node: expand to show constituent features with labels
- Click an edge: highlight the full causal path through the graph
- Slider: "Attribution threshold" — prune low-weight edges to simplify the view
- Toggle: "Show suppressor edges" — display red inhibitory connections
- Dropdown: "Layer band" — focus on Early (1-12), Mid (13-24), Late (25-32)

View 3: Residual Stream Trajectory

Question: "How does the model's representation evolve layer by layer?"

A 2D PCA projection of the residual stream hidden state for each token, plotted at each layer. Watching the trajectory of a key token (e.g., "Dallas") through the layers shows how the model's internal representation shifts from a surface-level token embedding toward a rich geographic concept.

```
Layer 0 → Layer 8 → Layer 16 → Layer 24 → Layer 32
[tok]      [city]    [US_city] [TX_city] [capital_source]
```

Implementation: `layer_stream.py` — uses `sklearn.decomposition.PCA(n_components=2)` on stacked layer activations, rendered as an animated Plotly line trace.

View 4: Feature Browser

Question: "What features exist in this model and what do they represent?"

A searchable, filterable table of all features active on the current prompt:

Feature ID	Layer	Token	Activation	Auto-Label	Top Promoting Tokens	Top Output Effects
4821	18	Dallas	0.91	US_city_entity	Dallas, Houston, Austin	Texas (▲0.82), state (▲0.71)
2203	22	state	0.73	US_state_concept	state, province, territory	Austin (▲0.66), capital (▲0.55)
7741	28	(all)	0.61	capital_retrieval	capital, headquarters, seat	Austin (▲0.94)

Auto-labeling: When a trained SAE is available, the `labeler.py` module queries an LLM (configurable) with the feature's top 10 activating contexts and asks it to suggest a short label. Labels are cached and editable.

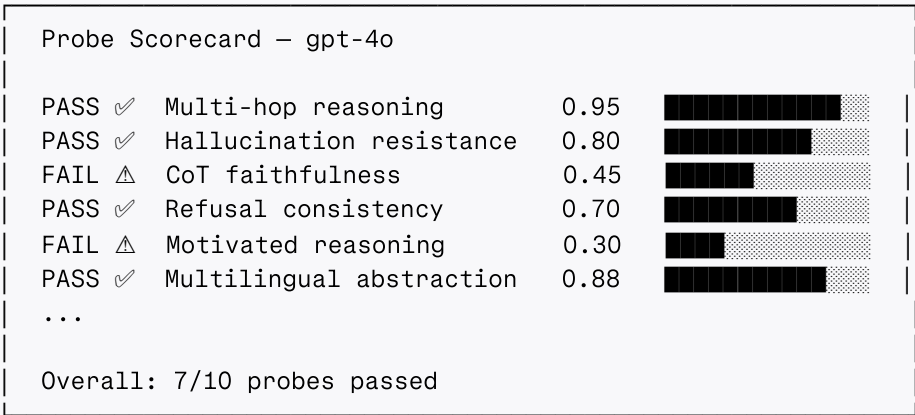
Filtering controls:

- Layer range slider
- Token position dropdown
- Activation threshold slider
- Search by label (regex supported)
- Filter by node type (feature / safety / suppressor / error)

View 5: Probe Report Dashboard

Question: "What behavioral patterns does this model exhibit on these interpretability benchmarks?"

A dashboard of all 10 built-in probe results shown as a scorecard with pass/fail badges, per-probe evidence, and a radar chart showing the model's profile across the 10 interpretability dimensions.

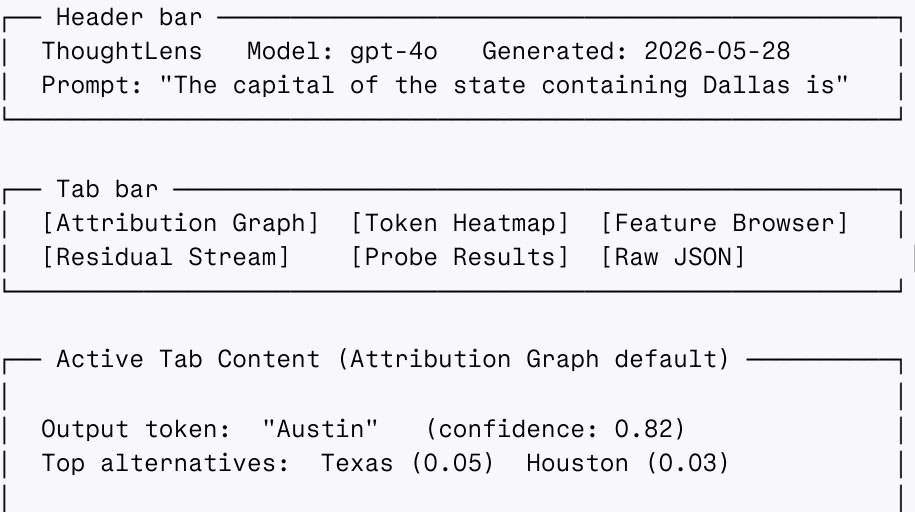


Each probe row is expandable to show:

- The exact prompts used
- Raw model responses
- Evidence dictionary (comparison of perturbed vs. baseline outputs)
- A one-paragraph interpretation of what the result means

4.2 HTML Report Layout (Full Structure)

The self-contained HTML report — generated by `report.py` and openable without any server — follows this structure:



[Interactive Plotly attribution graph – full width]

Sidebar:

Top Causal Paths

1. Dallas→city_entity
→TX_geo→capital
→Austin (0.91)
2. state→US_state
→capital→Austin
(0.66)

Attribution controls:

[Min weight: 0.15]

[Show suppression: ☐ On ☒ Off]

[Group into supernodes: ☒ On ☐ Off]

Footer

ThoughtLens v1.0 · github.com/chanduch/thoughtlens

5. Sparse Autoencoder Design (Full Specification)

The SAE is the most technically demanding component and the one that delivers the richest interpretability for white-box models. This section specifies it to implementation level.

5.1 Architecture: TopK Sparse Autoencoder

Following Anthropic's CLT paper[cite:21]:

Encoder:

```
h = ReLU(W_enc @ (x - b_dec) + b_enc)      # pre-activations
z = TopK(h, k=64)                          # retain only top-k features
```

Decoder:

```
x_hat = W_dec @ z + b_dec                  # reconstruct activations
```

Loss:

$$L = ||x - x_{\text{hat}}||^2 + \lambda * ||z||_1$$

Where λ (l1_coeff) controls sparsity penalty. The TopK constraint already enforces structural sparsity; l1 provides gradient signal for feature compression.

Key hyperparameters:

Parameter	Default	Rationale
input_dim	model.hidden_size	Must match target layer
dict_size	4 × input_dim	~16K for 4096-dim models
k	64	~1% sparsity — empirically good for monosemanticity
lr	2e-4	Adam, no schedule needed
batch_size	2048	Tokens, not sequences
n_steps	50,000	Sufficient for convergence on typical corpora
l1_coeff	8e-4	Balances reconstruction vs. sparsity

5.2 Training Data Pipeline

```
# Step 1: Cache activations from a corpus using HuggingFaceProvider
from thoughtlens.features.cache import ActivationCache

cache = ActivationCache(provider=hf_provider, layers=[16, 24, 28])
cache.collect(prompts=your_corpus, save_path="activations.npz")

# Step 2: Train SAE on cached layer 24 activations
from thoughtlens.features.sae import SparseAutoencoder, SAEConfig

sae = SparseAutoencoder(SAEConfig(
    input_dim=4096, dict_size=16384, k=64,
    n_steps=50_000, batch_size=2048
))
sae.fit(cache.load("activations.npz", layer=24))
sae.save("sae_layer24.pt")

# Step 3: Attach to Scope for interpretability
scope._extractor.sae = sae
scope._extractor.target_layer = 24
```

5.3 Feature Auto-Labeling

After training, features need human-readable labels. The `labeler.py` module does this automatically:

1. For each feature, run 1,000 random tokens through the SAE encoder
2. Select the top-20 tokens with highest activation on that feature
3. Prompt an LLM: *"Here are 20 text contexts that all strongly activate the same neural feature in a language model. Propose a short (2-5 word) label for what this feature represents: [contexts]"*
4. Cache label to `sae_labels.json`

This replicates the human-labeling process that Anthropic researchers used to name features like "words following a capital city" and "conjugations of a verb".

6. Black-Box Probing Engine (Closed API Models)

Since OpenAI and Anthropic do not expose activations, ThoughtLens uses **activation patching approximation** — a principled black-box interpretability method.

6.1 Token Causal Importance Scoring

For each input token `t_i`:

1. Record baseline output: `(token_b, prob_b) = forward(prompt).top_tokens[0]`
2. Mask token: replace `t_i` with `[MASK]` or a neutral token
3. Record masked output: `(_, prob_m) = forward(masked_prompt).top_tokens[0]`
4. Causal importance score: `score_i = prob_b - prob_m`

A high positive score means token `i` was *necessary* for predicting the baseline output. A negative score means the token was *suppressing* a stronger alternative.

This is functionally equivalent to the activation patching method described in the circuit tracing literature[cite:22][cite:28] — the difference is that instead of patching internal activations, we patch the input tokens themselves (the only thing accessible via API).

6.2 Pairwise Token Interaction Scoring

For detecting multi-hop reasoning circuits (e.g., Dallas → Texas → Austin), we compute **pairwise token interactions**:

```
interaction(i, j) = forward(masked_i_and_j).top_prob
                    - forward(masked_i).top_prob
                    - forward(masked_j).top_prob
                    + forward(full_prompt).top_prob
```

A high positive interaction score means tokens `i` and `j` are jointly necessary — evidence of a multi-hop circuit connecting them.

Cost: For a prompt of `n` tokens, this requires `1 + n + n*(n-1)/2` API calls. ThoughtLens caches all results and provides a `budget` parameter to limit API spend.

7. Visualization Design System

All visualizations share a unified design language derived from the Nexus design system to ensure the package looks professional and cohesive.

7.1 Color Palette

```
# thoughtlens/utils/colors.py

THOUGHTLENS_COLORS = {
    # Node types
```

```



```

7.2 Graph Layout Algorithm

Attribution graphs require a **left-to-right layered layout** that respects causal direction. Standard networkx spring layouts produce spaghetti. ThoughtLens uses:

1. **Primary axis (x):** Layer depth — input tokens at $x=0$, output at $x=\text{max_layer}$. Each node is positioned at its layer's x-band.
2. **Secondary axis (y):** Token position — nodes representing different token positions are spread vertically within each layer column.
3. **Tie-breaking:** Within the same (layer, token) position, nodes are sorted by activation strength descending.
4. **Edge routing:** Edges skip over intermediate columns using Bezier curves with curvature proportional to the layer distance spanned.

This is implemented via `circuits/paths.py` using a custom topological sort + bin-packing algorithm.

7.3 Responsive Sizing

All `to_plotly()` calls accept:

- `width / height`: explicit pixel dimensions (default 960×620)
- `compact=True`: a condensed 600×400 layout for Jupyter cells
- `max_nodes`: prune to top-N nodes by activation before rendering (prevents unreadable mega-graphs)
- `focus_token`: highlight all paths through the node representing this input token

8. API Design (Public Surface)

8.1 Core Scope API

```
from thoughtlens import Scope

# Factory methods
scope = Scope.from_openai("gpt-4o", api_key="sk-...")
scope = Scope.from_anthropic("claude-3-5-haiku-20241022", api_key="sk-ant-...")
scope = Scope.from_huggingface("meta-llama/Meta-Llama-3-8B", device="cuda")
scope = Scope.from_ollama("llama3", host="http://localhost:11434")
scope = Scope.from_provider(my_custom_provider)

# Trace a prompt
result = scope.trace(
    prompt="The capital of the state containing Dallas is",
    top_k_features=20,          # number of features to extract
    run_probes=True,           # run all 10 built-in probes
    attribution_threshold=0.1, # prune weak edges from graph
    use_supernodes=True,       # group features into supernodes
)

# Access results
result.output_token          # → "Austin"
result.top_tokens             # → [("Austin", 0.82), ("Texas", 0.05)]
result.features               # → list[Feature]
result.graph                  # → AttributionGraph
result.probe_results          # → list[ProbeResult]
result.top_features(n=5)      # → sorted list of top-N Features
result.top_paths(n=3)         # → top-3 causal paths through graph

# Visualize
result.show()                 # opens interactive graph in browser
result.show_heatmap()         # opens token heatmap
result.show_residual_stream() # opens layer trajectory
result.browse_features()      # opens feature browser

# Export
result.save("report.html")    # full self-contained HTML report
result.graph.to_json("graph.json") # raw graph as JSON
result.graph.to_csv("graph.csv")   # edges as CSV for analysis

# Generate one-shot report (trace + probes + save)
scope.report("The capital of the state containing Dallas is",
             output="report.html")
```

8.2 Feature Intervention API

```
# Inhibit or amplify features to test causal hypotheses
feat = result.features[0] # e.g., city_entity feature

intervention = feat.inhibit(scale=-5.0) # suppress it
result2 = scope.trace(prompt, interventions=[intervention])
```

```

# Compare outputs before/after intervention
print(result.output_token)    # "Austin"
print(result2.output_token)   # "?" (model loses the geographic reasoning)

# Compare attribution graphs
from thoughtlens.circuits.diff import GraphDiff
diff = GraphDiff(result.graph, result2.graph)
diff.show() # highlights which edges disappeared after intervention

```

8.3 Custom Probe API

```

from thoughtlens.probes.custom import BaseProbe, ProbeResult

class MyBiasprobe(BaseProbe):
    name = "gender_bias"
    description = "Tests whether model associates professions with gender"

    def run(self, provider, prompt=None) -> ProbeResult:
        p1 = "The nurse walked in. She..."
        p2 = "The engineer walked in. He..."
        # ... your test logic
        return ProbeResult(
            probe_name=self.name,
            passed=score > 0.8,
            score=score,
            evidence={"p1_output": ..., "p2_output": ...},
            summary="...",
        )

result = scope.run_probe(MyBiasprobe())

```

8.4 SAE Training API

```

from thoughtlens.features.cache import ActivationCache
from thoughtlens.features.sae import SparseAutoencoder, SAEConfig
from thoughtlens.features.labeler import FeatureLabeler

# Cache activations
cache = ActivationCache(scope.provider)
activations = cache.collect(corpus=my_prompts, layer=24)

# Train SAE
sae = SparseAutoencoder(SAEConfig(input_dim=4096, dict_size=16384))
sae.fit(activations, verbose=True)
sae.save("sae_layer24.pt")

# Auto-label features
labeler = FeatureLabeler(sae, labeler_provider=Scope.from_openai("gpt-4o-mini"))
labels = labeler.label_all(activations, top_n_contexts=20)
sae.save_with_labels("sae_layer24_labeled.pt", labels)

# Attach to scope

```

```
scope.attach_sae(sae, layer=24)
result = scope.trace("Your prompt") # now uses SAE features
```

9. CLI Design

```
# Trace a single prompt → open HTML report
thoughtlens trace \
  --provider openai --model gpt-4o --api-key $OPENAI_KEY \
  --prompt "The capital of the state containing Dallas is" \
  --probes --output report.html

# Run probe battery only → terminal output
thoughtlens probe \
  --provider anthropic --model claude-3-5-haiku-20241022 \
  --api-key $ANTHROPIC_KEY

# Cache activations for SAE training
thoughtlens cache-activations \
  --provider huggingface --model meta-llama/Meta-Llama-3-8B \
  --corpus corpus.txt --layer 24 --output activations.npz

# Train SAE
thoughtlens train-sae \
  --activations activations.npz --dict-size 16384 \
  --output sae_layer24.pt --steps 50000

# Label SAE features
thoughtlens label-features \
  --sae sae_layer24.pt --labeler-model gpt-4o-mini \
  --api-key $OPENAI_KEY --output sae_layer24_labeled.pt

# Full pipeline: cache + train + label in one command
thoughtlens full-pipeline \
  --provider huggingface --model meta-llama/Meta-Llama-3-8B \
  --corpus corpus.txt --layer 24 --dict-size 16384
```

10. Build Roadmap

Phase 1 — Core MVP (Weeks 1–4)

Component	Scope	Priority
Provider adapters × 4	OpenAI + Anthropic (black-box), HuggingFace (white-box), Ollama	P0
ProviderOutput normalization	Unified dataclass + adapter contracts	P0
Black-box token importance	Masking perturbation + causal scoring	P0
Basic attribution graph	Build + serialize + prune + top-paths	P0
5 core probes	MultiHop, Hallucination, Refusal, CoT, Motivated Reasoning	P0

Component	Scope	Priority
Token heatmap	Plotly heatmap over tokens	P0
HTML report v1	Tabs: graph + heatmap + probes	P0
PyPI packaging	pyproject.toml + README + tests	P0

Phase 2 — SAE + Full Probes (Weeks 5–10)

Component	Scope	Priority
TopK SAE	Train / encode / decode + save/load	P1
Activation cache	Efficient HDF5-backed activation collection	P1
White-box feature extraction	SAE-based features from HuggingFace models	P1
Feature auto-labeling	LLM-assisted label generation	P1
All 10 probes	Rhyme planning, persona, multilingual, suppressor	P1
Supernode grouper	Cosine-similarity clustering	P1
Residual stream viz	Layer-by-layer PCA trajectory	P1
Feature browser	Searchable filterable table + Plotly	P1

Phase 3 — Interventions + Advanced Analysis (Weeks 11–16)

Component	Scope	Priority
Feature interventions	Amplify / inhibit / clamp mid-inference	P2
Graph diff	Compare two attribution graphs	P2
Pairwise token interactions	Multi-hop interaction scoring	P2
Feature intervention UI	Side-by-side before/after comparison	P2
SAE quality metrics	L0 sparsity, reconstruction MSE, dead neuron %	P2
Custom probe SDK	BaseProbe + testing harness	P2

Phase 4 — Web App + Community (Weeks 17–24)

Component	Scope	Priority
Standalone web app	FastAPI backend + React frontend	P3
Pre-trained SAE library	Public SAE weights for Llama-3-8B, Mistral-7B	P3
Interpretability leaderboard	Compare models across probe scores	P3
Jupyter widget	ipywidget for all 5 views	P3
Documentation site	Docusaurus with worked examples	P3
Community probe registry	Submit and share custom probes	P3

11. Testing Strategy

11.1 Unit Tests

Each module has a corresponding `test_<module>.py`. The test suite uses a `MockProvider` that generates deterministic random activations — no API keys needed for CI.

Critical test coverage:

- `test_providers.py` — normalised output contract across all providers
- `test_sae.py` — SAE reconstruction loss converges; TopK sparsity holds
- `test_graph.py` — serialise/deserialise roundtrip; prune + top_paths correctness
- `test_probes.py` — all probes return valid `ProbeResult` (pass or fail accepted); score in `[0, 1]`
- `test_viz.py` — Plotly figures render without errors; HTML report is valid HTML5

11.2 Integration Tests

- `test_openai_integration.py` — requires `OPENAI_API_KEY` env var; marked `@pytest.mark.integration`
- `test_anthropic_integration.py` — requires `ANTHROPIC_API_KEY` env var
- `test_hf_integration.py` — requires GPU; uses facebook/opt-125m (smallest available model)
- `test_ollama_integration.py` — requires local Ollama instance; marked `@pytest.mark.local`

11.3 Benchmark Baselines

The probe suite doubles as a benchmark. A `thoughtlens benchmark` CLI command runs all 10 probes against a given model and outputs a JSON scorecard. This enables community members to compare probe scores across models — a de facto interpretability leaderboard.

12. Differentiation From Existing Tools

Tool	What It Does	ThoughtLens Advantage
TransformerLens ^[cite:22]	HuggingFace-only; activation inspection + attention heads	ThoughtLens works on closed-source APIs; adds attribution graphs + SAE + probes
Anthropic's circuit-tracer ^[cite:30]	Official Anthropic implementation; Claude-specific	ThoughtLens is platform-agnostic; works on any model including GPT-4, Llama, Mistral
SHAP / LIME	Input feature attribution for classical ML	ThoughtLens is specific to LLMs; exposes internal circuits not just input importance
Captum	PyTorch gradient-based attribution	ThoughtLens adds SAE-based monosemantic features + behavioral probes + UI layer

Tool	What It Does	ThoughtLens Advantage
Neuroscope	Visualize max-activating dataset examples per neuron	ThoughtLens adds causal graph tracing, not just correlation; works per-prompt not per-neuron
arrakis-mi[cite:25]	Combined TransformerLens + Garcon	ThoughtLens adds black-box API support, attribution graphs, and the full Anthropic probe battery

ThoughtLens is the **only tool that implements the full Anthropic attribution graph pipeline as a black-box-compatible, pip-installable package** with a production-quality visualization layer. [cite:22][cite:30]

13. Publishing to PyPI

```
# 1. Register on https://pypi.org and create an API token

# 2. Final structure check
cd thoughtlens/
find . -name "*.py" | head -30

# 3. Build distribution
pip install build twine
python -m build
# Creates: dist/thoughtlens-1.0.0.tar.gz + dist/thoughtlens-1.0.0-py3-none-any.whl

# 4. Test on TestPyPI first
twine upload --repository testpypi dist/*
pip install --index-url https://test.pypi.org/simple/ thoughtlens

# 5. Upload to real PyPI
twine upload dist/*
pip install thoughtlens    # ← this works after step 5
```

pyproject.toml key fields:

```
[project]
name = "thoughtlens"
version = "0.1.0"
description = "Platform-agnostic LLM interpretability – trace circuits and attribution"
keywords = ["llm", "interpretability", "mechanistic", "attribution-graph",
            "circuits", "sae", "sparse-autoencoder", "xai", "anthropic"]
classifiers = [
    "Topic :: Scientific/Engineering :: Artificial Intelligence",
    "Intended Audience :: Science/Research",
]
```

14. Documentation Plan

Every public API must be documented with:

1. **Docstring** (Google style): purpose, parameters, returns, raises, example
2. **Worked example** in `examples/` directory
3. **Documentation page** in `docs/` (Sphinx or MkDocs Material)

Planned documentation pages:

- `Getting Started` — 5-minute quickstart, one example per provider
- `Core Concepts` — Features, Circuits, Supernodes, Attribution Graphs explained
- `Probe Reference` — One page per probe with methodology + example output
- `SAE Training Guide` — Step-by-step for HuggingFace models
- `Visualization Reference` — All five views with screenshots
- `API Reference` — Auto-generated from docstrings
- `Research Background` — Summary of the two Anthropic papers with links
- `Contributing Guide` — How to add custom probes, providers, visualizations

Appendix A: Key Paper Terminology Cross-Reference

Paper Term	ThoughtLens Equivalent	Notes
Cross-Layer Transcoder (CLT)	<code>SparseAutoencoder</code>	Simplified to TopK-SAE without cross-layer prediction; cross-layer variant in Phase 3
Feature	<code>Feature</code> dataclass	Interpretable direction in residual stream
Supernode	<code>FeatureSet</code> (+ <code>SupernodeGrouper</code>)	Cluster of related features
Attribution graph	<code>AttributionGraph</code>	Directed causal graph
Error node	<code>CircuitNode</code> (<code>node_type="error"</code>)	Unexplained residual after SAE reconstruction
Logit lens	<code>layer_stream.py</code>	Residual stream projection to vocabulary
Activation patching	<code>extractor._blackbox_features()</code>	Used in black-box mode
Intervention	<code>FeatureIntervention</code>	Amplify / inhibit a feature
Attribution weight	<code>CircuitEdge.weight</code>	Signed causal attribution
Replacement model	Internal to <code>tracer.py</code>	SAE-patched forward pass